

DAY 4 BACKEND TASKS

D. Vishnu Vardhan Reddy

Donapativishnu21@gmail.com

Task 1 — User Registration API

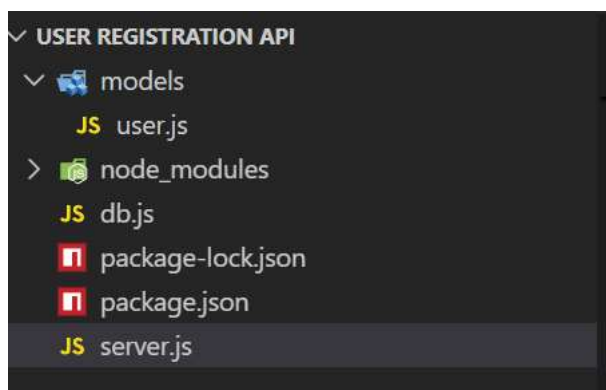
The User Registration API is implemented to allow new users to create accounts securely in the backend system. A POST `/register` API is created using Express.js to receive user information from the client. The API accepts three important fields: `name`, `email`, and `password`. These details are sent through the request body and processed by the server.

After receiving the user details, the server stores the data in MongoDB using Mongoose. Before saving the password, `bcryptjs` is used to hash the password for security purposes. Password hashing converts the original password into an encrypted format, which prevents storing plain text passwords in the database and improves user account protection.

Once the password is successfully hashed, the user data is stored securely in MongoDB. After successful registration, the API returns a success response such as "User registered successfully". This registration process improves authentication security and ensures safe user data management in the backend application.

VS CODE:

Structure



DAY 4 BACKEND TASKS

```
JS server.js > ...
1  const express = require("express");
2  const bcryptjs = require("bcryptjs");
3
4  require("./db");
5
6  const User = require("./models/User");
7
8  const app = express();
9
10 app.use(express.json());
11
12
13 // Home API
14 app.get("/", (req, res) => {
15   res.send("API is working");
16 });
17
18
19 // Register API
20 app.post("/register", async (req, res) => {
21
22   const { name, email, password } = req.body;
23
24   // Empty fields validation
25   if (!name || !email || !password) {
26     return res.json({
27       message: "All fields are required"
28     });
29   }
30
31   // Password length validation
32   if (password.length < 6) {
33     return res.json({
34       message: "Password must be at least 6 characters"
35     });
36   }
37
38   // Duplicate email check
39   const existingUser = await User.findOne({ email });
40
41   if (existingUser) {
42     return res.json({
43       message: "Email already registered"
44     });
45   }
46 }
```

DAY 4 BACKEND TASKS

```
45     }
46
47     // Hash password
48     const hashedPassword = await bcryptjs.hash(password, 10);
49
50     // Save user
51     const user = new User({
52       name,
53       email,
54       password: hashedPassword
55     });
56
57     await user.save();
58
59     res.json({
60       message: "User registered successfully"
61     });
62   });
63
64
65
66 // Login API
67 app.post("/login", async (req, res) => {
68
69   const { email, password } = req.body;
70
71   // Check user
72   const user = await User.findOne({ email });
73
74   if (!user) {
75     return res.json({
76       message: "User not found"
77     });
78   }
79
80   // Compare password
81   const isMatch = await bcryptjs.compare(
82     password,
83     user.password
84   );
85
86   if (!isMatch) {
87     return res.json({
```

```
88     message: "Invalid Password"
89   });
90   }
91
92   res.json({
93     message: "Login successful"
94   });
95
96 });
97
98
99 // Get All Users API
100 app.get("/users", async (req, res) => {
101
102   const users = await User.find();
103
104   res.json(users);
105
106 });
107
108
109 // Delete User API
110 app.delete("/user/:id", async (req, res) => {
111
112   await User.findByIdAndDelete(req.params.id);
113
114   res.json({
115     message: "User deleted successfully"
116   });
117
118 });
119
120
121 // Server
122 app.listen(5000, () => {
123   console.log("Server is running on port 5000");
124 });
```

Db.js

DAY 4 BACKEND TASKS



```
JS db.js  X  JS server.js  JS user.js

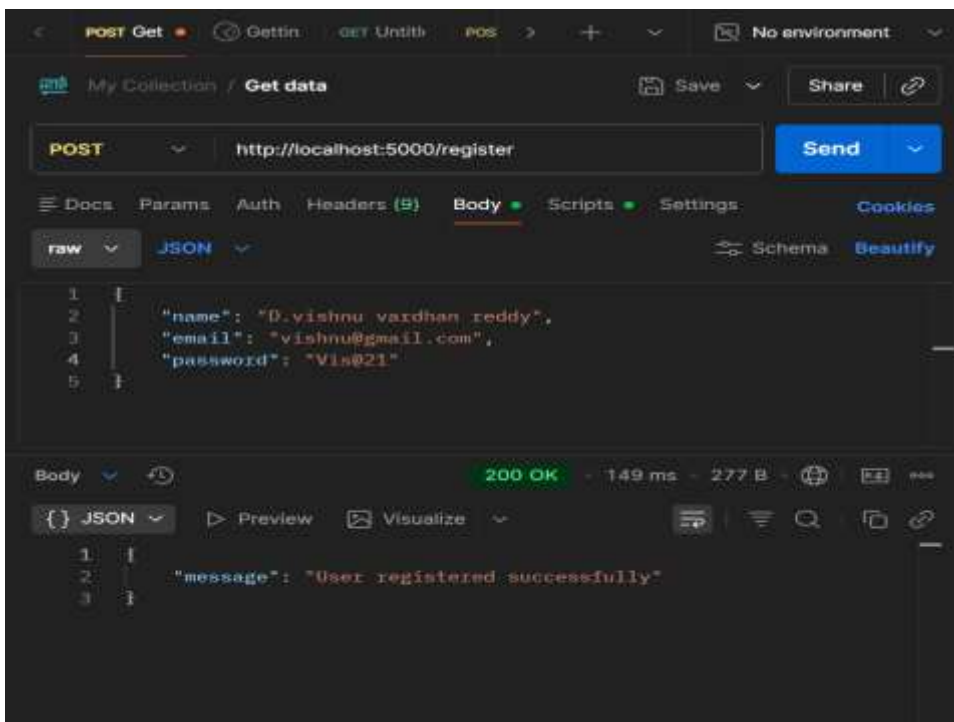
JS db.js > ...
1  const mongoose = require("mongoose");
2
3  mongoose.connect("mongodb://localhost:27017/authdb")
4  .then(() => console.log("MongoDB Connected Successfully"))
5  .catch((err) => console.log(err));
```

User.js



```
models > JS user.js > ...
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema({
4    name: String,
5    email: String,
6    password: String
7  });
8
9  module.exports = mongoose.model("User", userSchema);
```

POST/registerAPI



POST http://localhost:5000/register

Body (JSON):

```
{
  "name": "D.vishnu vardhan reddy",
  "email": "vishnu@gmail.com",
  "password": "Vis@21"
}
```

Response (200 OK):

```
{
  "message": "User registered successfully"
}
```

Along with their Emails and Password And connected to Mongo DB

DAY 4 BACKEND TASKS

Task 2 — User Login API

The User Login API is implemented to authenticate registered users in the backend system. A POST `/login` API is created using Express.js to accept user login credentials. The API receives the following fields from the request body:

- Email
- Password

When the login request is sent, the server first checks whether the entered email exists in MongoDB using `User.findOne({ email })`.

If the user does not exist:

User Not Found

If the email exists, the entered password is compared with the hashed password stored in the database using `bcryptjs.compare()`.

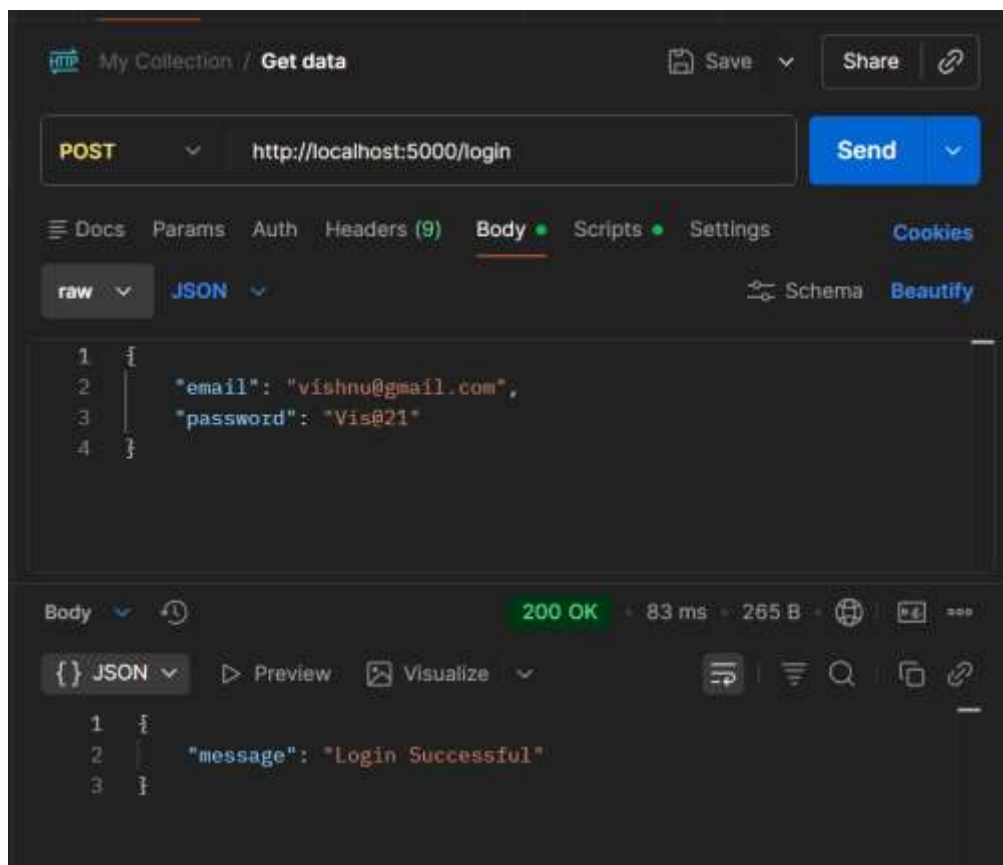
If the password matches:

Login Successful

If the password is incorrect:

Invalid Password

This login process helps provide secure authentication and protects user passwords using bcrypt encryption before storing them in MongoDB.



DAY 4 BACKEND TASKS

Checking whether the user exists in MongoDB.

```
// Check user
const user = await User.findOne({ email });

if (!user) {
  return res.json({
    message: "User not found"
  });
}
```

`User.findOne()` is used to search MongoDB for a user with the entered email address. It checks whether the email exists in the database. If the user exists, it returns the user data; otherwise, it returns `null`.

During registration, the user's password is encrypted (hashed) using `bcryptjs` before storing it in MongoDB. This improves security and prevents storing passwords in plain text format.

During login, the entered password is compared with the stored hashed password using `bcryptjs.compare()`.

Authentication Process

- If both passwords match → **Login Successful**
- If passwords do not match → **Invalid Password**

This process helps maintain secure user authentication and protects sensitive user passwords in the database.

`bcryptjs.compare()`

```
// Compare password
const isMatch = await bcryptjs.compare(
  password,
  user.password
);

if (!isMatch) {
  return res.json({
    message: "Invalid Password"
  });
}

res.json({
  message: "Login successful"
});
});
```

If both passwords match:

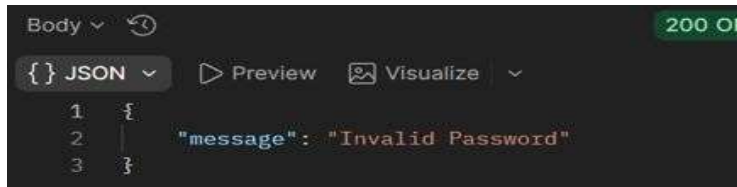
DAY 4 BACKEND TASKS

→ Login Successful

If passwords do not match:

→ Invalid Password

This helps keep user passwords secure and prevents storing plain text passwords in the database



Task 3 — User Management APIs

The User Management APIs are implemented to manage registered users in the backend system. Two APIs are created for this task:

- GET /users
- DELETE /user/:id

The GET /users API is used to fetch all registered users stored in MongoDB. When this API is called, the server retrieves all user records from the database using `User.find()` and returns the data in JSON format.

The DELETE /user/:id API is used to remove a user from MongoDB using the user's unique ID. The ID is passed through the API URL using `req.params.id`. The server uses `findByIdAndDelete()` to locate and delete the corresponding user record from the database.

After successful deletion, the API returns the message:

```
User deleted successfully
```

Both APIs are tested using Postman to verify proper API functionality and database operations. These APIs help in viewing and managing registered users efficiently in the authentication system.

DAY 4 BACKEND TASKS

The screenshot shows a REST client interface with a dark theme. At the top, there's a tab bar with 'GET Get c', 'Gettin', 'GET Untitl', and 'POS'. Below this, the URL bar shows 'http://localhost:5000/users' with a 'Send' button. The 'Body' tab is selected, showing a JSON response. The response status is '200 OK' with a response time of '9 ms' and a size of '894 B'. The JSON data is displayed in a code editor with line numbers 1 through 13.

GET ▼ http://localhost:5000/users Send ▼

Docs Params Auth Headers (9) **Body** Scripts Settings Cookies

raw ▼ **JSON** ▼ Schema Beautify

Body ▼ ↺ 200 OK • 9 ms • 894 B • 🌐 e.g. ⋮

{} **JSON** ▼ ▶ Preview 🖼️ Visualize ▼ ↻ ☰ 🔍 📄 🔗

```
1  [
2    {
3      "_id": "6a059ab5480a95c39c26450b",
4      "name": "D.vishnu vardhan reddy",
5      "email": "vishnu@gmail.com",
6      "password": "$2b$10$azC6M1c4gNzY0mYzSm5P70t9mmTNDZ/
7      zIkWgfHjkZtLIAtHtqGdrW",
8      "__v": 0
9    },
10   {
11     "_id": "6a059d1e480a95c39c26450c",
12     "name": "vignesh",
13     "email": "vignesh@gmail.com",
14     "password": "$2b$10$KbGpm8wUJGZ0KGDBGgJ9r0EWCvCp0cB2XtI4d0I9A3rxyy
15     .. 752"
```

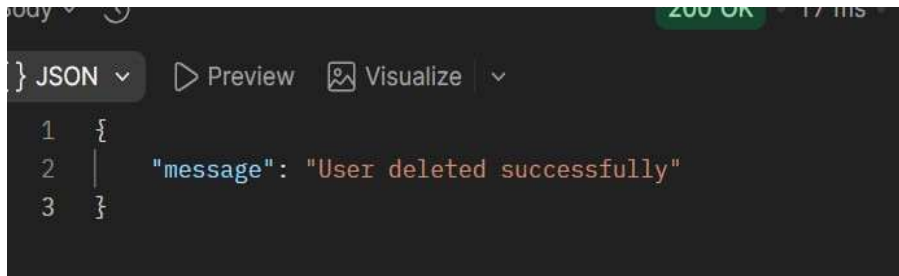

DAY 4 BACKEND TASKS

```
Body 200 OK • 9 ms • 894 B
JSON Preview Visualize
{
  "sNO.Z5C",
  ".__v": 0
},
{
  "_id": "6a059d3c480a95c39c26450d",
  "name": "yashwik",
  "email": "yashwik@gmail.com",
  "password": "$2b$10$zjVh4Q0PVgxLKEylzRw8e.gH64xpy62f0FvRi.iz2x70nqVKBvX8m",
  ".__v": 0
},
{
  "_id": "6a059d59480a95c39c26450e",
  "name": "rama",
  "email": "rama@gmail.com",
```

```
{
  "_id": "6a059d59480a95c39c26450e",
  "name": "rama",
  "email": "rama@gmail.com",
  "password": "$2b$10$Ap0yzVLUwF/jz5E4cyAe0ugp3PU1sVV9r/pJTv8fvmJaKxl6NpYWe",
  ".__v": 0
}
```

```
HTTP My Collection / Get data Save Share
DELETE http://localhost:5000/user/user_id_here Send
Docs Params Auth Headers (9) Body Scripts Settings Cookies
raw JSON Schema Beautify
1 {
2   "name": "rama",
3   "email": "rama@gmail.com",
4   "password": "rama898d#"
5 }
```

DAY 4 BACKEND TASKS



Task 3 — User Management APIs

In `server.js`, the `GET /users` API is implemented to retrieve all registered users stored in MongoDB. When this API is called, the server uses `User.find()` to fetch all user records from the database. The retrieved data is returned as a JSON response containing user details such as name, email, and password. This API helps administrators view and manage all registered users in the authentication system.

The `DELETE /user/:id` API is implemented to delete a specific user from MongoDB using the user's unique ID. The user ID is passed through the API URL using `req.params.id`. The server uses `findByIdAndDelete()` to locate and remove the corresponding user record from the database.

After successful deletion, the API returns:

```
User deleted successfully
```

If an invalid user ID is provided, no user will be deleted from the database. Both GET and DELETE APIs are tested using Postman to ensure proper API functionality and successful database operations. These APIs improve user management by allowing administrators to view and remove registered users efficiently.

Task 4 — Validation & Security

Validation and security features are implemented in `server.js` to improve authentication security and maintain proper user data in MongoDB.

Duplicate email validation is added inside the `/register` API using `User.findOne({ email })`. Before storing a new user, the server checks whether the entered email already exists in the database. If the email is already registered, the registration process is stopped and the API returns:

```
Email already registered
```

This helps maintain unique user accounts and prevents duplicate registrations.

Empty field validation is also implemented to ensure that users provide all required details such as name, email, and password. If any field is missing or empty, the API returns:

```
All fields are required
```

This improves data accuracy and prevents incomplete information from being stored in MongoDB.

DAY 4 BACKEND TASKS

Password validation is added to improve account security. Before saving user data, the server checks whether the password contains at least 6 characters. If the password is too short, the API returns:

```
Password must be at least 6 characters
```

This prevents weak passwords and improves authentication security.

Proper error handling is implemented throughout the authentication system. During login, if the entered email does not exist in MongoDB, the API returns:

```
User not found
```

If the entered password does not match the hashed password stored in the database, the API returns:

```
Invalid Password
```

These validation and error messages help users understand the exact issue and improve the usability of the authentication system.
